



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C8

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Hermetyzacja

Hermetyzacja (enkapsulacja) to jedna z podstawowych zasad programowania obiektowego, która polega na ukrywaniu szczegółów implementacji klasy i udostępnianiu tylko niezbędnych interfejsów do interakcji z obiektami tej klasy. Dzięki hermetyzacji możemy chronić dane przed nieautoryzowanym dostępem i modyfikacją.

Przykład 1. Klasa z hermetyzacją

W przykładzie klasy KontoBankowe mamy prywatne pole saldo, które jest dostępne tylko poprzez publiczne metody Wplata, Wyplata i PobierzSaldo. Dzięki temu możemy kontrolować, jak saldo jest modyfikowane, co zapobiega nieautoryzowanym zmianom.

```
using System;

public class KontoBankowe
{
    private decimal saldo;

    public KontoBankowe(decimal poczatkoweSaldo)
    {
        saldo = poczatkoweSaldo;
    }

    public void Wplata(decimal kwota)
    {
        if (kwota > 0)
        {
            saldo += kwota;
            Console.WriteLine($"Wpłacono: {kwota:C}. Nowe saldo: {saldo:C}");
        }
    }

    public void Wyplata(decimal kwota)
```

```

    {
        if (kwota > 0 && kwota <= saldo)
        {
            saldo -= kwota;
            Console.WriteLine($"Wypłacono: {kwota:C}. Nowe saldo: {saldo:C}");
        }
        else
        {
            Console.WriteLine("Niewystarczające środki na koncie.");
        }
    }

    public decimal PobierzSaldo()
    {
        return saldo;
    }
}

public class Program
{
    public static void Main()
    {
        KontoBankowe konto = new KontoBankowe(1000);
        konto.Wplata(500);
        konto.Wypłata(200);
        Console.WriteLine($"Aktualne saldo: {konto.PobierzSaldo():C}");
    }
}

```

Prywatne pole saldo: Jest dostępne tylko wewnątrz klasy KontoBankowe.

Publiczne metody Wplata, Wypłata i PobierzSaldo: Umożliwiają kontrolowany dostęp do pola saldo.

Polimorfizm

Polimorfizm to zdolność obiektów do przyjmowania różnych form. W programowaniu obiektowym polimorfizm pozwala na używanie tej samej metody w różnych kontekstach, co umożliwia bardziej elastyczne i rozszerzalne projektowanie kodu. Polimorfizm może być osiągnięty poprzez dziedziczenie i interfejsy.

Przykład 2: Polimorfizm z metodami wirtualnymi

W przykładzie klasy Zwierze oraz jej podklasy Pies i Kot mamy metodę WyдайDźwięk, która jest zdefiniowana w klasie bazowej i nadpisywana w klasach pochodnych.

```

using System;
using System.Collections.Generic;
public class Zwierze
{
    public virtual void WyдайDźwięk()
    {
        Console.WriteLine("Zwierzę wydaje dźwięk.");
    }
}

public class Pies : Zwierze
{
    public override void WyдайDźwięk()
    {
        Console.WriteLine("Pies szczeka.");
    }
}

public class Kot : Zwierze
{
    public override void WyдайDźwięk()
    {
        Console.WriteLine("Kot miauczy.");
    }
}

public class Program

```

```

{
    public static void Main()
    {
        List<Zwierze> zwierzeta = new List<Zwierze>
        {
            new Pies(),
            new Kot(),
            new Zwierze()
        };

        foreach (var zwierz in zwierzeta)
        {
            zwierz.WydajDzwiek();
        }
    }
}

```

Klasa Zwierze: Definiuje metodę WydajDzwiek, która jest wirtualna (virtual), co oznacza, że może być nadpisana w klasach pochodnych.

Klasy Pies i Kot: Nadpisują (override) metodę WydajDzwiek, aby dostarczyć specyficzne dla siebie implementacje.

Lista zwierzeta: Przechowuje obiekty różnych typów (Pies, Kot, Zwierze), ale wszystkie są traktowane jako obiekty typu Zwierze. Dzięki polimorfizmowi możemy wywołać metodę WydajDzwiek na każdym z tych obiektów, a odpowiednia implementacja zostanie użyta w zależności od rzeczywistego typu obiektu.

Zadania do samodzielnego rozwiązania

Zadanie 1. Utwórz klasę Samochod z prywatnymi polami marka, model i rokProdukcji. Dodaj publiczne metody do ustawiania i pobierania wartości tych pól oraz metodę WyświetlInformacje(), która wyświetla wszystkie informacje o samochodzie.

Zadanie 2. Rozbuduj klasę KontoBankowe, dodając metodę Przelew(KontoBankowe inneKonto, decimal kwota), która pozwala na przelew środków między kontami.

Zadanie 3. Utwórz klasę Figura z metodą ObliczPole(). Następnie utwórz klasy Kwadrat i Kolo, które dziedziczą po klasie Figura i implementują metodę ObliczPole() odpowiednio dla kwadratu i koła.

Zadanie 4. Utwórz klasę Instrument z metodą Gra(). Następnie utwórz klasy Gitara i Pianino, które dziedziczą po klasie Instrument i implementują metodę Gra() odpowiednio dla gitary i pianina.